

8. What is the main purpose of a destructor?

- a) To copy objects
- b) To allocate memory
- c) To free memory and clean resources
- d) To overload operators

Answer: c) To free memory and clean resources

II. ONE WORD ANSWER QUESTIONS (1 MARKS):

1. How does an inline constructor differ from a normal constructor?

Answer: It is defined inside the class body.

2. What is the main idea of using the + operator in operator overloading?

Answer: To add or combine user-defined objects easily.

3. How does a programmer declare a binary operator overloading function in C++?

Answer: return_type operator+(ClassName obj);

4. What is the main idea of defining operator-()?

Answer: To implement unary negation, which changes the sign of an object's value.

5. How does a class declare a friend function?

Answer: By using the friend keyword inside the class definition.

6. Why ~ (tilde) symbol is used in destructor?

Answer: To indicate that the function is used to destroy an object and free memory.

7. How does the C++ compiler respond when a programmer tries to overload the scope resolution (::) operator?

Answer: It gives a compilation error because the :: operator cannot be overloaded.

8. Why semicolon (;) is used in C++?

Answer: To properly terminate statements and avoid compilation errors.

III. SHORT ANSWER QUESTIONS (3 MARKS):

1. Explain the constructor and provide an example?

Answer:

- Constructors are special member function that are automatically called whenever an object of a class is created, whose task is to initialize the objects of its class.
- Its name is the same as the class name.
- The constructor is invoked whenever an object of its associated class is created.

Example:

```
class integer                                // Class with a constructor
{
    int m;
public:
    integer(void);                          // Constructor declaration
};
integer :: integer(void)                    // Constructor definition
{
    m = 0;
}
```

2. Explain the concept of a default constructor and illustrate it with an example?

Answer:

- A constructor that accepts no parameters is called the default constructor.
- A default constructor is automatically created by the compiler if no constructor is defined.
- It takes no arguments and initializes members with default values.

General syntax:

ClassName::ClassName()

Example:

```
class integer
{
    int m;
public:
    integer()                // default constructor
    {
        m = 0;
    }
};
int main()
{
    integer I1;
    return 0;
}
```

3. Explain the role of Parameters in a Constructor?

Answer:

- Parameters in a constructor are used to receive values and initialize the data members of a class. Constructor parameters can be of any data type.
- A constructor cannot accept an object of its own class by value because it leads to infinite constructor calls. Therefore, an object of the same class must be passed by reference to avoid recursion.

Example:

```
class A
{
public:
    A(A&); // Valid
};
```

4. Explain the role of a destructor in memory management.

Answer:

- A destructor is a special member function of a class that is automatically called when an object goes out of scope.
- The main role of a destructor in memory management is to release the memory and system resources that were allocated to an object during its lifetime.
- It helps in cleaning up dynamic memory, closing files, and freeing other resources, thereby preventing memory wastage and memory leaks in a program.

5. What is meant by an inline parameterized constructor? Explain its purpose.

Answer:

- An inline parameterized constructor is a constructor that is defined inside the class and takes parameters to initialize data members of an object.
- It is called automatically when an object is created with values passed as arguments.
- The main purpose of an inline parameterized constructor is to initialize object data members at the time of object creation and to improve program execution speed by reducing function call overhead.

6. Explain the concept of unary operator overloading?

Answer:

- Unary operator overloading is the process of defining an operator function so that a unary operator can operate on a user-defined data type.
- A unary operator works on only one operand, and when overloaded, it performs the specified operation on the calling object.
- When overloaded using a member function, the operator function takes no arguments because the object is passed implicitly.

General syntax:

Declaration(inside the class):

```
return_type operator op();
```

Definition syntax (outside the class):

```
return_type ClassName :: operator op()
{
    // statements
}
```

Example:

```
class number
{
    int x;
public:
    void operator-();
};

void number::operator-()
{
    x = -x;
}
```

7. Explain the difference between a constructor and a destructor.

Answer:

Constructor	Destructor
A constructor is used to initialize the object and allocate the required resources.	A destructor is used to clean up the object and deallocate the resources.
The name of the constructor is the same as the class name.	The name of the destructor is the same as class name but is preceded by a tilde (~).
A constructor can accept parameters and can be overloaded.	A destructor cannot accept parameters and cannot be overloaded.
A constructor is called automatically when an object is created.	A destructor is called automatically when an object is destroyed.
A class can contain more than one constructor.	A class can contain only one destructor.

8. Explain operator overloading and describe its syntax.

Answer:

- Operator overloading allows an operator to perform different operations depending on the operands used.
- It is implemented using a special function called an operator function.

General syntax:

Declaration (inside the class):

```
return_type operator operator_symbol(argumentlist);
```

Definition (outside the class):

```
return_type classname :: operator operator_symbol (argumentlist)
{
    // Function body
}
```

Where:

- return_type specifies the type of value returned.
- classname is the name of the class.
- op is the operator being overloaded.
- operator op is operator function name

IV. LONG ANSWERS QUESTIONS (5 MARKS):

1. Describe a parameterized constructor with example.

Answer:

- A parameterized constructor is a constructor that takes one or more arguments.
- It allows different objects of the same class to be initialized with different values at the time of creation.
- Unlike a default constructor, a parameterized constructor requires the programmer to pass values while creating objects.
- Objects can call the constructor either implicitly or explicitly.

Example:

```
class Integer
{
    int m;
public:
    Integer(int x)                // Parameterized constructor
    {
        m = x;
    }
};
int main()
{
    Integer int1(100);           // Implicit call
    Integer int2 = Integer(75);  // Explicit call
    return 0;
}
```

2. Describe a copy constructor with example.

Answer:

- A copy constructor is a special constructor used to create a new object by copying the values of an existing object of the same class.
- It is used to initialize a new object with the data members of an existing object, rather than assigning values manually.
- A copy constructor takes a reference to an object of the same class as its parameter.
- **General Syntax:**
ClassName(ClassName &object);

Example:

```
class Integer
{
    int m;
```

```

public:
    Integer(int x)                // Parameterized constructor
    {
        m = x;
    }
    Integer(Integer &obj)         // Copy constructor
    {
        m = obj.m;
    }
};
int main()
{
    Integer A(10);                // Parameterized constructor called
    Integer B(A);                 // Copy constructor called
    return 0;
}

```

3. Explain constructor overloading with an example?

Answer:

- Constructor overloading is the concept in which more than one constructor is defined within the same class.
- Each constructor differs in its number of parameters, type of parameters, or order of parameters.
- A class may contain a constructor with no arguments, a constructor with parameters, and a copy constructor.
- When multiple constructors exist in a class, they are said to be overloaded constructors.

Example:

```

class integer
{
    int m;
public:
    integer()                // default constructor
    {
        m = 0;
    }
    integer(int a)           // parameterized constructor
    {
        m = a;
    }
    integer(integer &i)       // copy constructor
    {
        m = i.m;
    }
};
int main()
{
    integer I1;              // calls no-argument constructor
    integer I2(20);          // calls parameterized constructor
    integer I3(I2);          // calls copy constructor
    return 0;
}

```

4. **Explain constructor with default arguments with an example.**

Answer:

- A constructor with default arguments is a constructor in which one or more parameters are assigned default values.
- If no values are passed during object creation, the default values are used automatically.
- If values are passed, they replace the default values.
- Default values are allowed only for the last parameters of the constructor.
- Such a constructor can also work as a default constructor when no arguments are passed.

Example:

```
class complex
{
    float real, image;
public:
    complex(float r, float i = 0)
    {
        real = r;
        image = i;
    }
    void display()
    {
        cout << real << " + " << image << "i" << endl;
    }
};

int main()
{
    complex C1(5.0);    // default value used for image
    complex C2(2.0, 3.0); // default value overridden
    C1.display();
    C2.display();
    return 0;
}
```

5. **Describe the role of a destructor in a class.**

Answer:

- A destructor is a special member function that is used to destroy objects created by a constructor.
- Like a constructor, the destructor has the same name as the class, but it is preceded by a tilde (~).
- Destructor never takes any arguments.
- It does not return any value.
- Destructor is invoked automatically by the compiler when an object goes out of scope.
- It is used to clean up system resources.
- **General Syntax:**
~Classname()

Example:

```
class book
{
public:
    book()                                // Constructor
    {
        cout << "Book object created" << endl;
    }
}
```

```

    ~book()                                // Destructor
    {
        cout << "Book object destroyed" << endl;
    }
};
int main()
{
    book b;                                // Object creation
    return 0;                              // Destructor called automatically
}

```

6. Explain overloading binary operators using a member function with a relevant example.

Answer:

- Overloading a binary operator is the process of creating a special operator function that allows a binary operator to perform operations on two objects of a user-defined class.

- General syntax:**

Declaration inside the Class:

```
return_type operator operator_symbol (class_name obj);
```

Definition Outside the Class:

```
return_type class_name :: operator operator_symbol (class_name parameter)
{
    // statements
}
```

Example:

```

class Demo
{
    int num;
public:
    Demo(int n)                                // Parameterized constructor
    {
        num = n;
    }
    Demo operator+(Demo obj);                  // Declaration of overloading + operator within the class
};

Demo Demo::operator+(Demo obj)                // Declaration of overloading + operator outside the class
{
    return Demo(num + obj.num);
}

int main()
{
    Demo d1(10);
    Demo d2(20);
    Demo d3 = d1 + d2;                          // Using overloaded '+' operator
    return 0;
}

```

7. Explain overloading binary operators using a friend function with a relevant example.

Answer:

- Friend function is also used to overload binary operators.
- It is a special function defined outside the class that can perform operations on two objects of the class.
- A friend function is not a member of the class but can access private members.
- For binary operators, both operands are passed as arguments to the friend function.

• **General Syntax:**

Declaration inside the class:

```
friend return_type operator op(classname, classname);
```

Definition outside the class:

```
return_type operator operator_symbol (class_name obj1, class_name obj2)
{
    // statements
}
```

Example:

```
class Demo
{
    int num;
public:
    Demo(int n)
    {
        num = n;
    }

    friend Demo operator+(Demo, Demo);           // Friend function declaration
};
Demo operator+(Demo d1, Demo d2)                // Friend function definition
{
    return Demo(d1.num + d2.num);
}
int main()
{
    Demo d1(10);
    Demo d2(20);
    Demo d3 = d1 + d2;                           // Calls friend operator function
    return 0;
}
```

8. Explain how to manipulate strings using operator overloading with an example.

Answer:

- Operator overloading allows strings to be combined using operators like +.
- Normally, combining strings requires extra steps, but overloading makes it simple.
- A class can define how operators work on its objects, so strings can behave like numbers.
- In the example, the class has a string variable and overloads + to join two strings.
- input() takes input from the user, show() displays the string, and the overloaded + operator returns a new object with the combined string.
- The statement s3 = s1 + s2; creates a new string object containing both strings joined, which is then displayed using s3.show().

Example:

```
class String
{
    char str[20];
public:
    void input()
    {
        cout << "Enter string: ";
        cin >> str;
    }
    void show()
    {
        cout << str << endl;
    }
    String operator+(String s)           // Overloading '+' operator to join two strings
    {
        String result;
        strcpy(result.str, str);         // Copy first string
        strcat(result.str, s.str);       // Add second string
        return result;
    }
};
int main()
{
    String s1, s2, s3;
    s1.input();                          // First string
    s2.input();                          // Second string
    s3 = s1 + s2;                        // Join strings using overloaded '+'
    cout << "Concatenated String: ";
    s3.show();

    return 0;
}
```

OUTPUT:

```
Enter string: Hello
Enter string: World
Concatenated String: HelloWorld
```

7. What is the role of the base class constructor when a derived class object is created in C++?

- a) Initializes base class members first
- b) Initializes derived class members first
- c) Only derived class constructor is called
- d) No constructor is called

Answer: a) Initializes base class members first

8. What is the purpose of the protected access specifier in C++ inheritance?

- a) It allows access only within the same function
- b) It allows access only outside the class
- c) It allows access within the class and derived classes
- d) It prevents access everywhere

Answer: c) It allows access within the class and derived classes

II. ONE WORD ANSWER QUESTIONS (1 MARKS)

1. How would you declare a function prototype before calling it in a C++ program?

Answer: return_type function_name(argument-list);

2. What OOP feature would you use to reuse properties of an existing class in a new class?

Answer: Inheritance

3. What technique would you use to define multiple functions with the same name but different parameters?

Answer: Overloading

4. What inheritance type would you apply to create a chain of derived classes from a base class?

Answer: Multilevel inheritance

5. What problem would occur when duplicate members are inherited from the same base class in multiple inheritance?

Answer: Ambiguity

6. What keyword would you use to prevent duplicate copies of base class members in multiple inheritance?

Answer: Virtual

7. What would you use in a derived class to initialize both base class and derived class members?

Answer: Constructor

8. What keyword would you use to create a function that must be overridden in a derived class?

Answer: pure virtual (used with =0)

III. SHORT ANSWER QUESTIONS (3 MARKS)

1. How would you use visibility mode to control access of base class members in a derived class?

Answer:

Visibility modes in inheritance (public, protected, private) control how base class members are accessible in the derived class.

Public inheritance: Public members remain public, and protected members remain protected in the derived class.

Protected inheritance: Public and protected members become protected in the derived class.

Private inheritance: Public and protected members become private in the derived class. Private members of the base class cannot be accessed directly in the derived class.

Thus, visibility modes help control the level of access and maintain data security in inheritance.

2. How would you design a program using multiple inheritance to combine features of two classes?

Answer:

Multiple inheritance is a feature of C++ in which a derived class inherits properties and functions from more than one base class. This helps to combine the features of multiple classes into a single class.

Syntax:

```
class Derived : public Base1, public Base2
```

```
{  
};
```

Example:

```
class A
```

```
{
```

```
public:
```

```
    void showA()
```

```
{
```

```
    cout<<"Class A function"<<endl;
```

```
}
```

```
};
```

```
class B
```

```
{
```

```
public:
```

```
    void showB()
```

```
{
```

```
    cout<<"Class B function"<<endl;
```

```
}
```

```
};
```

```
class C : public A, public B
```

```
{
```

```
};
```

```
int main()
```

```
{
```

```
    C obj;
```

```
    obj.showA();
```

```
    obj.showB();
```

```
}
```

3. What approach would you use to solve ambiguity in multiple inheritance?

Answer:

Cause of Ambiguity:

Occurs when two or more base classes have functions with the same name. The compiler cannot decide which function to call from the derived class.

Resolution:

Use scope resolution operator (::) to specify which base class function to call.

Example:

```
class A
{
public:
    void show()
    {
        cout << "Class A"<<endl;
    }
};
class B
{
public:
    void show()
    {
        cout << "Class B"<<endl;
    }
};
class C: public A, public B
{
public:
    void show()
    {
        A::show();                // resolves ambiguity
    }
};
int main()
{
    C obj;
    obj.show();
}
```

4. What example would you use to implement hierarchical inheritance in a real program?**Answer:**

Hierarchical inheritance occurs when more than one derived class inherits from a single base class. The members of the base class are shared by all the derived classes.

Example: (Bank System)

```
#include<iostream>
using namespace std;
class Account
{
public:
    void show()
    {
        cout<<"Account details"<<endl;
    }
};
class SavingsAccount : public Account
{
public:
```

```

void savings()
{
    cout<<"Savings Account"<<endl;
}
};
class CurrentAccount : public Account
{
public:
    void current()
    {
        cout<<"Current Account"<<endl;
    }
};
int main()
{
    SavingsAccount s;
    CurrentAccount c;
    s.show();          // inherited from Account
    c.show();          // inherited from Account
}

```

5. How would you apply abstract base class concept in a program?

Answer:

An abstract base class is a class that cannot be instantiated and contains at least one pure virtual function (= 0). The derived class must implement the pure virtual function.

Example:

```

#include<iostream>
using namespace std;
class Shape                                // Abstract base class
{
public:
    virtual void draw() = 0;                // pure virtual function
};
class Circle : public Shape                // Derived class
{
public:
    void draw()
    {
        cout << "Drawing Circle\n";
    }
};
int main()
{
    Circle c; // create object of derived class
    c.draw(); // call implemented function
    return 0;
}

```

Shape is an abstract base class, and Circle implements the pure virtual function draw().

6. What would you do to initialize base and derived class using constructors?

Answer:

In inheritance, the base class constructor is automatically called when an object of the derived class is created. The derived class constructor can initialize the base class using the base class constructor in the initializer list.

Syntax:

```
Derived(parameters) : Base(parameters)
{
}
```

Example:

```
class Base
{
public:
    Base(int x)
    {
        cout << "Base class constructor: " << x << endl;
    }
};
class Derived : public Base
{
public:
    Derived(int a, int b) : Base(a)
    {
        cout << "Derived class constructor: " << b << endl;
    }
};
int main()
{
    Derived d(10, 20);
    return 0;
}
```

7. How would you create a derived class from a base class and reuse its function?

Answer:

A derived class is created from a base class using inheritance. The derived class can reuse the public and protected functions of the base class.

Syntax:

```
class Derived : public Base
{
};
```

Example:

```
class Base
{
public:
    void show()
    {
        cout << "This is base class function" << endl;
    }
};
```

```

class Derived : public Base
{
};
int main()
{
    Derived d;
    d.show();           // reusing base class function
    return 0;
}

```

8. How would you implement hybrid inheritance in a simple program?

Answer:

Hybrid inheritance is a combination of two or more types of inheritance, such as multiple inheritance and hierarchical inheritance, in a single program.

Example:

```

#include<iostream>
using namespace std;
class A
{
public:
    void showA()
    {
        cout << "Class A"<<endl;
    }
};
class B : public A
{
public:
    void showB()
    {
        cout << "Class B"<<endl;
    }
};
class C : public A
{
public:
    void showC()
    {
        cout << "Class C"<<endl;
    }
};
class D : public B, public C
{
public:
    void showD()
    {
        cout << "Class D"<<endl;
    }
};

```

```

int main()
{
    D obj;
    obj.showB();
    obj.showC();
    obj.showD();
    return 0;
}

```

IV. LONG ANSWERS QUESTIONS (5 MARKS)

1. What is meant by function overloading? Explain the different ways to overload a function. Illustrate with an example program.

Answer:

Function overloading is a feature of C++ that allows two or more functions to have the same name, but they must differ in the number of arguments or the data type of arguments. The compiler determines which function to execute based on the arguments passed during the function call. This improves code readability and flexibility.

Function overloading can be done in the following ways:

1. By changing the number of arguments

In this method, functions have the same name but differ in the number of parameters.

Example:

```

int sum(int x, int y)
{
    return x + y;
}
int sum(int x, int y, int z)
{
    return x + y + z;
}
int main()
{
    cout << sum(10, 20) << endl;           // Calls sum with 2 arguments
    cout << sum(10, 20, 30) << endl;       // Calls sum with 3 arguments
    return 0;
}

```

2. By changing the data type of arguments

In this method, functions have the same name and same number of parameters, but the data types of parameters are different.

Example:

```

int sum(int x, int y)
{
    return x + y;
}
double sum(double x, double y)
{
    return x + y;
}

```

```

int main()
{
    cout << sum(10, 20) << endl;    // Calls integer version
    cout << sum(10.5, 20.5) << endl; // Calls double version
    return 0;
}

```

2. What is meant by inheritance? Illustrate any two types of inheritance with simple examples.

Answer:

Inheritance is the mechanism in C++ by which a new class is created from an existing class. The existing class is called the base class, and the new class is called the derived class. The derived class inherits the properties and functions of the base class.

1. Single Inheritance

If a class is derived from a single base class, it is called as single inheritance.

Example:

```

class Person
{
public:
    void display()
    {
        cout << "Person details" << endl;
    }
};
class Student : public Person
{
};
int main()
{
    Student s;
    s.display();
    return 0;
}

```

2. Multilevel Inheritance

When a class is derived from another derived class, it is called multilevel inheritance.

Example:

```

class A
{
public:
    void showA()
    {
        cout << "Class A" << endl;
    }
};
class B : public A
{
public:

```

```

void showB()
{
    cout << "Class B" << endl;
}
};
class C : public B
{
};
int main()
{
    C obj;
    obj.showA();
    obj.showB();
    return 0;
}

```

3. Explain "public" and "protected" visibility modes with suitable examples.

Answer:

Public Inheritance:

In public inheritance:

- Public members of the base class remain public in the derived class.
- Protected members remain protected in the derived class.
- Private members cannot be accessed directly in the derived class.

Example:

```

class A
{
public:
    int x;
protected:
    int y;
};
class B : public A
{
};
int main()
{
    B obj;
    obj.x = 10; // Allowed
    cout << obj.x;
    return 0;
}

```

Protected Inheritance:

In protected inheritance:

- Public members of the base class become protected in the derived class.
- Protected members remain protected.
- Private members cannot be accessed.

Example:

```
class A
{
public:
    int x;
};
class B : protected A
{
public:
    void setValue()
    {
        x = 20; // Allowed inside derived class
        cout << x;
    }
};
int main()
{
    B obj;
    obj.setValue();
    return 0;
}
```

4. How virtual base class avoids duplication of base class members. Illustrate with an example.**Answer:**

In multiple inheritance, a derived class may inherit the same base class through more than one path. This creates duplicate copies of base class members and causes ambiguity.

This problem can be avoided by declaring the base class as virtual.

Example:

```
class student
{
public:
    int roll;
};
class test : virtual public student
{
};
class sports : virtual public student
{
};
class result : public test, public sports
{
};
int main()
{
    result r;
    r.roll = 101;
    cout << "Roll number = " << r.roll<<endl;
    return 0;
}
```

5. Explain how a derived class can inherit from multiple base classes. Write a C++ program to demonstrate it.

Answer:

In multiple inheritance, a derived class inherits members from more than one base class using comma-separated base classes.

Example:

```
#include <iostream>
using namespace std;
class student
{
public:
    int roll;
};
class marks
{
public:
    int score;
};
class result : public student, public marks
{
public:
    void display()
    {
        cout << "Roll number = " << roll << endl;
        cout << "Score = " << score << endl;
    }
};
int main()
{
    result r;
    r.roll = 101;
    r.score = 90;
    r.display();
    return 0;
}
```

6. Illustrate nesting of classes with a suitable example.

Answer:

Nesting of classes, also known as containership, means a class contains objects of other classes as its data members instead of using inheritance. In this method, one class makes use of another class by creating its object inside it.

When an object of the containing class is created, the objects of the contained classes are created first using their constructors. After that, the constructor of the containing class is executed. To initialize the nested objects, a constructor initialization list is used.

Example:

```
class A
{
public:
    class B                // Nested class
    {
    public:
        void show()
        {
            cout << "This is nested class"<<endl;
        }
    };
};
int main()
{
    A::B obj;              // Access nested class
    obj.show();
    return 0;
}
```

7. Explain the role of function prototyping in C++ with an example.**Answer:**

Function prototyping is a declaration statement that describes the function interface to the compiler by specifying the number and type of arguments and the return type. It helps the compiler ensure that correct arguments are passed to the function and detects errors at compilation time.

Function prototype is written as:

return_type function_name(argument-list);

Example:

```
#include <iostream>
using namespace std;
float volume(int x, float y, float z); // function prototype
int main()
{
    float v;
    v = volume(2, 3.5, 4.5); // function call
    cout << "Volume = " << v;
    return 0;
}
float volume(int a, float b, float c) // function definition
{
    float v;
    v = a * b * c;
    return v;
}
```

8. Illustrate how function overloading improves program flexibility using a suitable example.

Answer:

Function overloading allows defining multiple functions with the same name but different number or types of arguments. The compiler automatically selects the correct function based on the arguments passed.

Function overloading improves program flexibility, code maintenance, and easier interface between programs and real-world objects.

Example:

```
int product(int p, int q, int r)
{
    return p * q * r;
}

float product(float x, float y, float z)
{
    return x * y * z;
}

int main()
{
    cout << "Integer product = " << product(2, 3, 4) << endl;
    cout << "Float product = " << product(2.5f, 3.5f, 4.5f);
    return 0;
}
```

7. What would happen if a base class pointer points to a derived class object and the function is virtual?

- a) Base version executes
- b) Derived version executes
- c) Both execute
- d) None

Answer: b) Derived version executes

8. What would you include to prevent a class from being instantiated but allow inheritance?

- a) Static constructor
- b) Friend function
- c) Pure virtual function
- d) Inline function

Answer: c) Pure virtual function

II. ONE WORD ANSWER QUESTIONS (1 MARKS)

1. Predict the output if the base class function is not declared virtual and is called using a base class pointer referring to a derived object.

Answer: The base class function will execute due to early binding.

2. Consider the following class:

```
class Shape {  
public:  
    virtual void display() = 0;  
};
```

What will happen if you try to create an object of Shape in main()?

Answer: A compilation error will occur because Shape is an abstract class and cannot be instantiated.

3. What would happen if an object is created for a class that contains a pure virtual function?

Answer: A compilation error will occur because abstract classes cannot be instantiated.

4. How would you enable a base class pointer to refer to a derived class object in a program?

Answer: Inheritance must be used so that the base class pointer can store the address of a derived class object.

5. Predict what happens when a derived class overrides a virtual function and it is called using a base class pointer.

Answer: The derived class function executes due to runtime polymorphism.

6. How would you store the address of a Derived class object in a base class pointer? Write a small code snippet to demonstrate this.

Answer:

```
class Base  
{  
};  
class Derived : public Base  
{  
};
```

```
int main()
{
    Derived obj;
    Base *ptr = &obj;
}
```

7. **How would you dynamically create an object of class Student using a pointer and store its address?**

Answer: `Student *ptr = new Student();`

8. **Predict what would happen if dynamic memory allocated using new is not released using delete.**

Answer: Memory leak will occur because the allocated memory is not freed.

III. SHORT ANSWER QUESTIONS (3 MARKS)

1. **How you would use a pointer to access members of an object.**

Answer:

A pointer to an object stores the address of that object. It allows indirect access to data members and member functions. The arrow operator (->) is used to access members through a pointer. This approach is useful when objects are handled dynamically.

Example:

```
Student s;
Student *ptr = &s;
ptr->roll = 10;
Here, ptr accesses the object using the arrow operator.
```

2. **How would you use this pointer to resolve ambiguity in a class?**

Answer:

The this pointer refers to the calling object of a member function. It is mainly used when local variables and data members have the same name. Using this->variableName clearly refers to the data member of the class. This avoids confusion between local and instance variables.

Example:

```
void set(int x){
    this->x = x;
}
Here, this->x refers to the class data member.
```

3. **What approach would you use to achieve runtime polymorphism in C++?**

Answer:

To achieve runtime polymorphism, inheritance must be used. The base class function should be declared as virtual. The derived class must override the same function. A base class pointer should then refer to a derived class object to enable late binding.

Example:

```
Base *ptr;
Derived obj;
ptr = &obj;
This allows function call to be decided at runtime.
```

4. Predict what would happen if the virtual keyword is not used in overriding.

Answer:

If the virtual keyword is not used, early binding occurs. The base class function will execute even if the pointer refers to a derived class object. Runtime polymorphism will not work properly. The function call will be decided at compile time.

Thus, virtual is necessary for late binding.

5. How dynamic memory allocation is used to create an array of objects.

Answer:

Dynamic memory allocation allows objects to be created at runtime using the new operator. When creating multiple objects, memory can be allocated in the form of an array. The pointer stores the base address of the first object. This is useful when the number of objects is decided during execution.

Example:

```
Item *ptr;
```

```
ptr = new Item[3];
```

This creates three objects dynamically.

6. What would you do if you want to prevent a class from being instantiated?

Answer:

To prevent a class from being instantiated, at least one pure virtual function must be declared in the class. This makes the class an abstract class. Abstract classes cannot have objects of their own. However, they can be used as base classes for inheritance.

Example:

```
virtual void show() = 0;
```

7. How function overriding supports runtime polymorphism.

Answer:

Function overriding occurs when a derived class provides its own definition of a base class function. The function must have the same name and parameters. If the base function is declared virtual, the derived version executes at runtime. This supports dynamic behaviour in programs.

Example:

```
class Derived : public Base
```

```
{
    void show()
{
    cout<<"Derived";
}
};
```

8. What changes would you make to convert early binding into late binding?

Answer:

To convert early binding into late binding, the base class function must be declared using the virtual keyword. This enables the compiler to use dynamic dispatch. The function call will then be resolved at runtime. This is essential for achieving runtime polymorphism.

Thus, adding virtual enables late binding.

IV. LONG ANSWERS QUESTIONS (5 MARKS)

1. How runtime polymorphism is implemented using virtual functions in C++.

Answer:

Runtime polymorphism allows a program to decide which function to execute at runtime. It is implemented using inheritance and virtual functions. The base class function must be declared as virtual. The derived class overrides the function. A base class pointer is used to refer to the derived class object. The function call is resolved at runtime using late binding.

Example:

```
#include<iostream>
using namespace std;
class Base
{
public:
virtual void show()
{
cout << "Base class function";
}
};
class Derived : public Base
{
public:
void show()
{
cout << "Derived class function";
}
};
int main()
{
Base *ptr;
Derived obj;
ptr = &obj;
ptr->show();
return 0;
}
```

2. Design a program using an abstract base class and explain how it enforces implementation in derived classes.

Answer:

An abstract base class contains at least one pure virtual function. It cannot be instantiated directly. It provides a common interface for derived classes. Every derived class must implement the pure virtual function. If not implemented, the derived class also becomes abstract. This ensures compulsory function definition in all child classes.

Example:

```
#include<iostream>
using namespace std;
class Shape
{
```

```

public:
virtual void draw() = 0;
};
class Circle : public Shape
{
public:
void draw()
{
cout << "Drawing Circle";
}
};
int main()
{
Circle obj;
obj.draw();
return 0;
}

```

3. How a base class pointer can be used to manage derived class objects dynamically.

Answer:

A base class pointer can store the address of a derived class object because of inheritance. This is known as upcasting. When the function is virtual, the derived class function executes at runtime. This technique is useful in handling multiple object types uniformly. It enables flexibility and extensibility in programs.

Example:

```

#include<iostream>
using namespace std;
class Base
{
public:
virtual void display()
{
cout << "Base display";
}
};
class Derived : public Base
{
public:
void display()
{
cout << "Derived display";
}
};
int main()
{
Base *ptr = new Derived();
ptr->display();
delete ptr;
return 0;
}

```

4. How an array of objects can be created dynamically using pointers.

Answer:

Dynamic memory allocation allows creation of objects at runtime. When multiple objects are needed, memory can be allocated in the form of an array using the new operator. The pointer stores the base address of the first object. Each object can then be accessed using indexing. This method provides flexibility in managing memory.

Example:

```
#include<iostream>
using namespace std;
class Item
{
public:
int code;
void set(int c)
{
code = c;
}
void show()
{
cout << code << " ";
}
};
int main()
{
Item *ptr = new Item[3];
for(int i=0;i<3;i++)
{
ptr[i].set(i+1);
}
for(int i=0;i<3;i++)
{
ptr[i].show();
}
delete[] ptr;
return 0;
}
```

5. How this pointer can be used to return the current object and support method chaining.

Answer:

this pointer holds the address of the calling object. It can be returned from a function to allow multiple function calls on the same object. This technique is called method chaining. It improves readability of the program. It ensures that operations are performed on the same object instance.

Example:

```
#include<iostream>
using namespace std;
class Test
{
int x;
public:
```

```

Test* set(int a)
{
this->x = a;
return this;
}
void show()
{
cout << x;
}
};
int main()
{
Test obj;
obj.set(10)->show();
return 0;
}

```

6. Demonstrate what would happen if the virtual keyword is removed from a polymorphic program.

Answer:

If the virtual keyword is removed, runtime polymorphism fails. The function call will be resolved at compile time. This is called early binding. Even if the base class pointer refers to a derived object, the base class function executes. Thus, dynamic behaviour is lost.

Example:

```

#include<iostream>
using namespace std;
class Base
{
public:
void show()
{
cout << "Base";
}
};
class Derived : public Base
{
public:
void show()
{
cout << "Derived";
}
};
int main()
{
Base *ptr;
Derived obj;
ptr = &obj;
ptr->show();
return 0;
}

```

7. How function overriding supports runtime polymorphism with proper example.

Answer:

Function overriding occurs when a derived class defines a function with the same name and signature as in the base class. When the base class function is virtual, the overridden function executes at runtime. This supports dynamic behaviour. It ensures correct function execution based on object type. Overriding is essential for runtime polymorphism.

Example:

```
#include<iostream>
using namespace std;
class Animal
{
public:
virtual void sound()
{
cout << "Animal sound";
}
};
class Dog : public Animal
{
public:
void sound()
{
cout << "Dog barks";
}
};
int main()
{
Animal *ptr;
Dog obj;
ptr = &obj;
ptr->sound();
return 0;
}
```

8. Design a program using pure virtual functions to enforce implementation in derived classes.

Answer:

A pure virtual function is declared using =0. It forces derived classes to implement that function. The base class becomes abstract. It provides a common structure for all derived classes. This ensures consistency and proper design in object-oriented programming.

Example:

```
#include<iostream>
using namespace std;
class Vehicle
{
public:
virtual void start() = 0;
};
```

```
class Car : public Vehicle
{
public:
void start()
{
cout << "Car starts";
}
};
int main()
{
Car obj;
obj.start();
return 0;
}
```

7. What can be passed by non-type template parameters during compile- time?

- a) int
- b) constant expression
- c) float
- d) char

Answer: b) constant expression

8. What approach would you use to include statements that may cause a program to terminate abnormally in C++ exception handling?

- a) Try block
- b) Catch block
- c) Finally block
- d) Throw block

Answer: a) Try block

II. ONE WORD ANSWER QUESTIONS (1 MARKS)

1. How would you use a template in C++ to create a family of related classes or functions?

Answer: Using Classes and functions

2. What can be passed by non-type template parameters during compile-time?

Answer: Constant expression

3. Predict the type of exception generated when an out-of-range index error occurs in a program.

Answer: An out-of-range index error produces a synchronous exception.

4. Predict what would happen if a function specifies throw(int, double) but throws a char exception.

Answer: If a function declared with throw(int, double) throws a char, the program terminates abnormally because the thrown type is not allowed by the specification.

5. What would happen if the catch(...) block is not placed at the end of the catch list?

Answer: If the catch(...) block is not placed at the end of the catch list, it will cause a compilation error.

6. What would you do if you want to prevent a function from throwing any exceptions in C++?

Answer: If you want to prevent a function from throwing any exceptions in C++, you should declare it with an empty exception specification.

7. What might you include in your explanation of class templates in C++ and how they help in creating generic class operations when only the data types differ?

Answer: Reusability

8. What might you include about template functions being overloaded by either template or ordinary functions in C++?

Answer: Overloading

III. SHORT ANSWER QUESTIONS (3 MARKS)

1. **How would you solve a problem of handling multiple data types using Class templates?**

Answer:

- Class templates are used to create generic classes that can work with different data types.
- When the same class logic is required for different data types, writing separate classes for each type increases code size and makes maintenance difficult.
- Class templates solve this problem by allowing a single class definition to be used for multiple data types, thus promoting code reuse and making programs efficient and easy to maintain.

2. **How would you define and use Member Function templates in C++?**

Answer:

- Member function templates are functions that belong to a template class and work with different data types.
- The member functions of a template class can be defined either inside or outside the class.
- When they are defined outside the class, they must be written using the template syntax.

The general form is:

```
template <class T>
return_type ClassName<T>::function_name(arguments)
{
    // Function body
}
```

Here, T represents the data type, and the function works according to that type.

3. **What approach would you use to categorize runtime errors in C++ as either internally generated or externally caused? Explain the types of exceptions.**

Answer:

Runtime errors in C++ are categorized as:

1. **Synchronous exceptions** – caused by internal program errors.
2. **Asynchronous exceptions** – caused by external system events.

Runtime errors are classified into two types based on their cause:

1. **Synchronous Exceptions**

- These are caused by errors inside the program, such as wrong input or array out-of-bounds.
- They are handled using C++ exception handling mechanisms (try, throw, catch).

2. **Asynchronous Exceptions**

- These are caused by external factors such as hardware failure, disk errors, or keyboard interrupts.
- They are not directly handled by C++ exception handling.

4. **What approach would you use to implement the throwing mechanism for error handling in program?**

Answer:

- The throwing mechanism is used to generate exceptions when an error occurs.
- An exception is thrown using the throw statement.
- The general forms are:

```
throw exception; // Throw an exception
throw;           // Re-throw an exception
```

- The exception can be of any data type such as int, char, float, or an object.

- When an exception is thrown, the program control immediately leaves the try block and is transferred to the corresponding catch block, where the error is handled.

5. How would you use the catching mechanism to handle exceptions in program?

Answer:

Code for handling exceptions is included in catch blocks. A catch block looks like a function definition:

```
catch(type arguments)
{
    // Statements for managing exceptions
}
```

The type indicates the type of exception that the catch block handles. The parameter 'arguments' is an optional parameter name. When the exception is caught, the code in the catch block is executed.

6. How you would implement a mechanism to catch all exceptions in a try-catch block.

Answer:

- catch(...) is a special catch block called the catch-all handler.
- It is used to catch all types of exceptions, regardless of their data type.
- It is useful when the type of exception is unknown.
- The catch(...) block must always be placed at the end of all catch blocks.

Example:

```
#include <iostream>
using namespace std;
int main()
{
    try
    {
        int x = 10;
        throw x;                // Throw integer exception
    }
    catch(...)                  // Catch-all handler
    {
        cout << "Exception caught" << endl;
    }
    return 0;
}
```

7. How you would declare a function template to perform operations on various data types.

Answer:

- A function template is declared using the keyword template followed by template parameters inside <>.
- It allows a single function to work with different data types.
- The keyword class or typename can be used to represent a template parameter.
- When the function is called with a specific data type, the compiler creates a separate version of the function for that type.

Example:

```
template <class T>    // Function Template
T max(T a, T b)
{
```

```

    return (a > b) ? a : b;
}

int main()
{
    int I1 = 10, I2 = 20;
    float F1 = 5.5, F2 = 3.2;
    max(I1, I2);           // For int
    max(F1, F2);           // For float
    return 0;
}

```

8. How would you use function template with multiple parameters while creating a program?

Answer:

A function template in C++ allows a function to work with different data types. It is also possible to use more than one generic data type in a function template. Multiple template parameters are specified in the template statement using a comma-separated list.

Syntax:

```

template <class T1, class T2>
returntype functionname(T1 arg1, T2 arg2)
{
    // function body
}

```

Here, T1 and T2 represent generic data types which are replaced by actual data types when the function is called.

Example:

```

template <class T1, class T2>
void display(T1 x, T2 y)
{
    cout << x << y << endl;
}

int main()
{
    display<int, float>(10, 25.5);           // Integer and Float
    display<char, int>('A', 100);           // Character and Integer

    return 0;
}

```

IV. LONG ANSWERS QUESTIONS (5 MARKS)

1. How would you explain a Class Template with multiple parameters in detail?

Answer:

A class template with multiple parameters allows a class to use more than one generic data type. It is useful when a class needs to store and process different types of data together. Instead of creating separate classes for different data types, a single template class can be used. This reduces code repetition and improves program maintainability.

Declaration Syntax:

```
template <class T1, class T2>
class ClassName
{
    // Class body
};
```

Example:

```
template <class T1, class T2>           // Class template with two parameters
class Test
{
    T1 a;
    T2 b;
public:
    Test(T1 x, T2 y)
    {
        a = x;
        b = y;
    }
};
int main()
{
    Test<int, int> T1(10, 20);
    Test<float, char> T2(3.5, 'A');
    T1.show();
    T2.show();
    return 0;
}
```

2. How you would overload a template function to handle multiple types or operations.**Answer:**

Template function overloading allows multiple functions with the same name to work with different data types or different numbers of parameters.

A template function can be overloaded in two main ways:

1. By creating another template function with the same name but different parameters.
2. By creating a normal (ordinary) function with the same name.

During function call, the compiler selects the most suitable function based on the arguments.

Example:

```
template <class T>
T add(T a, T b)
{
    return a + b;
}
template <class T>
T add(T a, T b, T c)
{
    return a + b + c;
}
```

3. How to rethrow an exception.

Answer:

Rethrowing an exception means sending an exception again after it has been caught.

It is used when a function cannot fully handle an exception and wants to pass it to a higher-level function.

In C++, an exception is rethrown using the statement:

throw;

This statement is written inside a catch block without any argument.

It sends the same exception to the next enclosing try-catch block.

Example:

```
#include <iostream>
using namespace std;
void check(int b)
{
    try
    {
        if (b == 0)
            throw 0;                // Throw int exception
    }
    catch(int x)
    {
        cout << "Error found in check() function" << endl;
        throw;                    // Re-throw
    }
}
int main()
{
    try
    {
        check(0);
    }
    catch(int y)
    {
        cout << "Handled finally in main()" << endl;
    }
    return 0;
}
```

4. How would you explain the exception handling mechanism with an example in C++?

Answer:

Exception handling is used to handle runtime errors and prevent abnormal program termination.

It is implemented using three keywords: try, throw, and catch.

- try block contains the code that may cause an error.
- throw is used to generate an exception.
- catch block is used to handle the exception.

This mechanism separates error-handling code from normal program code.

Example:

```

#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    try
    {
        if (b == 0)
        {
            throw b;                // Generate exception
        }
        cout << "Result = " << a / b << endl;
    }
    catch (int x)
    {
        cout << "Error: Division by zero is not allowed." << endl;
    }
    return 0;
}

```

5. How you would define and catch a specified exception in a try-catch block.**Answer:**

It is possible to restrict a function to throw only certain specified exceptions. This is achieved by adding a **throw list clause** to the function definition.

The general form of using an exception specification is:

```
type function(arglist) throw (type-list)
```

```

{
// Function body
}

```

The type-list specifies the types of exceptions that the function is allowed to throw.

If any other type is thrown, the program terminates abnormally.

An empty throw list means that the function will not throw any exception:

```
throw();
```

Example:

```

#include <iostream>
using namespace std;
void test(int x) throw(int, double)    // Function with specified exceptions
{
    if (x == 1)
        throw x;    // int exception
    else if (x == -1)
        throw 1.5;  // double exception
}

```

```

int main()
{
    try
    {
        test(1);
        test(-1);
    }
    catch (int)
    {
        // Handle int exception
    }
    catch (double)
    {
        // Handle double exception
    }
    return 0;
}

```

6. How would you explain a function template with multiple parameters in detail, and how is it used in C++?

Answer:

A function template with multiple parameters allows a function to work with **more than one data type** at the same time. It is useful when a function performs the same operation for different combinations of data types.

Multiple template parameters are declared using a comma-separated list inside the template statement.

General Syntax:

```

template <class T1, class T2>
return_type function_name(T1 a, T2 b)
{
    // Function body
}

```

T1 and T2 represent different data types that are decided at the time of function call.

Example:

```

#include <iostream>
using namespace std;
template <class T1, class T2>
void show(T1 a, T2 b)
{
    cout << a << " " << b << endl;
}
int main()
{
    show<int, float>(10, 20.5);
    show<char, int>('A', 100);
    return 0;
}

```

7. What approach would you use to declare a class template for different data types or operations?

Answer:

A class template is used when the same class logic is required for different data types. It allows the programmer to write a single class that can work with multiple data types.

A class template is declared using the template keyword with a generic data type.

General Syntax:

```
template <class T>
class ClassName
{
public:
    T data;
    void show();
};
```

T is a template parameter that represents any data type.

Example:

```
#include <iostream>
using namespace std;
template <class T>
class Sample
{
    T value;
public:
    Sample(T v)
    {
        value = v;
    }
    void display()
    {
        cout << "Value: " << value << endl;
    }
};

int main()
{
    Sample<int> obj1(10);
    Sample<float> obj2(5.5);
    obj1.display();
    obj2.display();

    return 0;
}
```

8. How would you use overloaded binary operators using friend function?

Answer:

A binary operator can be overloaded using a friend function when we want the operator function to access private data members and when the left-hand operand is not an object of the class.

A friend function is declared inside the class using the friend keyword and is defined outside the class. Unlike member functions, a friend function takes both operands as arguments.

General Syntax:

```
friend return_type operator+(ClassName, ClassName);
```

Example:

```
#include <iostream>
using namespace std;
class Number
{
    int value;
public:
    Number(int v = 0)
    {
        value = v;
    }
    friend Number operator+(Number n1, Number n2);    // Friend function declaration
    void display()
    {
        cout << value << endl;
    }
};
Number operator+(Number n1, Number n2)    // Friend function definition
{
    Number temp;
    temp.value = n1.value + n2.value;
    return temp;
}
int main()
{
    Number a(10), b(20), c;
    c = a + b;    // Calls operator+(a, b)
    c.display();
    return 0;
}
```